

Nettoyage des données

Analyse des données — Séance 3

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

1. **Reprise** : sélectionner, filtrer, vérifier. Vingt-cinq minutes.
2. Les valeurs manquantes : les trouver, et comprendre d'où elles viennent.
3. Décider quoi en faire — et assumer la décision.
4. Les doublons, qui ne sont pas tous des erreurs.

La première partie referme la séance précédente. La suite ouvre le vrai sujet.

**Reprise — ce qu'on n'a pas eu
le temps de voir**

Choisir des colonnes

```
df["POP"]                                # une colonne (Series)
df[["LIBGEO", "POP", "TAUX_CHOMAGE"]]    # plusieurs (DataFrame)
```

Les doubles crochets ne sont pas une coquetterie : vous passez une *liste* de noms. C'est la structure vue en séance 1.

```
df["CAT_URBAINE"].value_counts()
df["CAT_URBAINE"].value_counts(normalize=True)
```

`value_counts()` est le premier réflexe sur toute variable qualitative. Les quatre modalités affichées doivent correspondre à celles du dictionnaire — sinon, quelque chose ne va pas.

Filtrer des lignes

```
df[df["POP"] > 100000]
```

La condition à l'intérieur produit une suite de vrai/faux, un par ligne. Le tableau ne conserve que les lignes vraies.

```
df[(df["POP"] > 50000) & (df["TAUX_CHOMAGE"] > 18)]      # et  
df[(df["CAT_URBAINE"] == "1") | (df["CAT_URBAINE"] == "2")]  # ou  
df[df["CODGEO"].str[:2].isin(["75", "93", "95"])]          # appartenance
```

Les parenthèses autour de chaque condition sont obligatoires. Sans elles, Python évalue dans le mauvais ordre et renvoie une erreur peu explicite.

Trier, et compter

```
df.sort_values("NB_ENTREPRISES", ascending=False).head(10)
```

Trier par ordre décroissant et regarder les dix premières lignes est le moyen le plus rapide de repérer une anomalie. Ici, deux communes moyennes affichent 9 999 établissements : c'est un code de non-réponse, pas un effectif.

```
len(df)                # nb de lignes  
df["TAUX_CHOMAGE"].isna().sum()  # nb de valeurs manquantes
```

Comptez avant et après chaque filtrage. Un filtre qui supprime les trois quarts de l'échantillon signale généralement une erreur de condition, pas un fait économique.

Trois vérifications avant de quitter le fichier

1. `df.shape` correspond-il à ce qu'annonce la documentation ?
2. `df.info()` : chaque colonne a-t-elle le type attendu ?
3. `value_counts()` sur les variables qualitatives : les modalités correspondent-elles au dictionnaire ?

Ces trois contrôles prennent deux minutes. Ils vous éviteront la moitié des erreurs du semestre — et ils constituent une question type du contrôle continu.

Vous savez charger un fichier, comprendre ce qu'il contient, en extraire ce qui vous intéresse et vérifier que vous ne vous êtes pas trompée.

Il reste le travail que personne ne montre : **rendre ces données utilisables**.

Le principe de la suite

Le nettoyage n'est pas une opération technique préalable à l'analyse. C'est une suite de **décisions**, dont chacune doit pouvoir se justifier — et dont chacune modifie le résultat.

Les valeurs manquantes

```
df.isna().sum()                # effectif par colonne
df.isna().mean().sort_values() # part par colonne

df[df["salaire"].isna()][ "secteur"].value_counts()
```

La seconde ligne est la plus importante : elle demande **qui** sont les manquants.

Si les valeurs absentes se concentrent dans certains secteurs, certaines régions, certains niveaux de revenu, le problème n'est pas technique. Il est économique.

Les manquants qui ne ressemblent pas à des manquants

Codage	Exemple	Symptôme
Code numérique	9, 99, 999	Une moyenne d'âge à 47 ans
Texte	"nd", "NR", "- "	Colonne numérique lue en texte
Chaîne vide	" "	Comptée comme une modalité
Valeur impossible	-1, 0, 1900	Aucun symptôme

Pandas ne les reconnaît pas : `isna()` renvoie zéro manquant, et tous les calculs fonctionnent.

Seul le dictionnaire des variables vous dira comment la non-réponse est codée.

Les convertir

```
import numpy as np

df["age"] = df["age"].replace([99, 999], np.nan)
df["secteur"] = df["secteur"].replace(["nd", "NR", ""], np.nan)

# Verifier ce qu'on vient de faire
print(df["age"].isna().sum(), "manquants apres conversion")
print(df["age"].describe())
```

Chaque conversion se vérifie immédiatement. Une moyenne d'âge qui passe de 47 à 41 ans confirme que les 99 étaient bien des non-réponses — et mesure au passage l'ampleur de l'erreur évitée.

Trois origines, trois conséquences

Manquant complètement au hasard

L'absence ne dépend de rien. Un questionnaire égaré, une ligne corrompue.

Supprimer les lignes réduit la précision, mais ne biaise pas.

Manquant au hasard, conditionnellement

L'absence dépend de variables *observées*. Les jeunes répondent moins.

Supprimer biaise ; on peut corriger en tenant compte de l'âge.

Manquant non au hasard

L'absence dépend de la valeur *elle-même*. Les hauts revenus ne déclarent pas leur revenu.

Aucune correction n'est possible à partir des seules données observées.

Sur les variables de revenu et de patrimoine, la non-réponse est presque toujours du troisième type.

- Les très hauts revenus déclarent moins souvent.
- Les revenus non déclarés sont donc, en moyenne, plus élevés.
- Supprimer ces lignes sous-estime la moyenne *et* les inégalités.

On ne peut pas trancher à partir des données seules : par construction, on ne voit pas ce qui manque.

Ce qu'on peut faire, c'est **le dire**, et mesurer la sensibilité du résultat à l'hypothèse retenue.

Quand l'absence est une information

Une valeur manquante n'est pas toujours un trou à combler. Elle peut être une donnée à part entière.

Trois exemples

Un montant de prime vide : la personne n'en a peut-être pas touché.

Une date de fin de contrat vide : le contrat est peut-être toujours en cours.

Un revenu du patrimoine vide : le ménage n'en a peut-être aucun.

Dans les trois cas, remplacer par la moyenne serait absurde — et supprimer la ligne éliminerait précisément la population qui vous intéresse.

Seul le dictionnaire tranche entre « non renseigné » et « sans objet ». Ce sont deux choses différentes.

Décider

Trois options, aucune neutre

Option	Ce qu'on fait	Ce que cela coûte
Supprimer les lignes	on retire les observations incomplètes	échantillon réduit, biais possible
Imputer	on remplace par une valeur	fausse précision
Conserver et signaler	on ne touche à rien, on rapporte les effectifs	analyse plus lourde

Signaler n'est pas une action sur les données : c'est une action sur le compte rendu. On laisse les manquants en place, les calculs les ignorent, et on dit combien d'observations ont réellement servi.

C'est le cas général — et la seule option qui ne suppose rien. Supprimer suppose que les manquants sont sans structure ; imputer suppose qu'on sait par quoi les remplacer.

Supprimer : ce qui disparaît vraiment

```
avant = len(df)
df_propre = df.dropna(subset=["salaire", "age", "diplome"])
print(avant, "->", len(df_propre))
```

La suppression porte sur les lignes ayant *au moins un* manquant parmi les variables retenues. Avec cinq variables incomplètes à 5 % chacune, on peut perdre un quart de l'échantillon.

Le contrôle à faire

Comparez les caractéristiques des lignes supprimées à celles des lignes conservées. Si elles diffèrent, votre échantillon final ne représente plus la même population.

Le piège de l'indexation chaînée

```
df[df["POP"] < 2000]["TAUX_CHOMAGE"] = 0
```

```
SettingWithCopyWarning: A value is trying to be set on  
a copy of a slice from a DataFrame
```

Deux crochets successifs produisent une *copie* : la modification s'applique à la copie, puis disparaît. Aucune erreur, aucun effet.

```
df.loc[df["POP"] < 2000, "TAUX_CHOMAGE"] = 0
```

.loc désigne lignes et colonnes en une seule opération. **Pour modifier, c'est toujours .loc.**

Châîner plutôt qu'empiler

```
# Empile des variables intermediaires
df1 = df[df["CAT_URBAINE"] == "1"]
df2 = df1[["LIBGEO", "POP", "TAUX_CHOMAGE"]]
df3 = df2.sort_values("TAUX_CHOMAGE", ascending=False)

# Chaîne : une seule expression, lisible de haut en bas
resultat = (
    df
    .loc[df["CAT_URBAINE"] == "1",
          ["LIBGEO", "POP", "TAUX_CHOMAGE"]]
    .sort_values("TAUX_CHOMAGE", ascending=False)
)
```

Les parenthèses extérieures autorisent le retour à la ligne. C'est l'idiome courant de pandas : une opération par ligne, dans l'ordre où on les pense.

Supprimer une ligne, ou supprimer une colonne

```
df.dropna(subset=["salaire"])      # retire des OBSERVATIONS  
df.drop(columns=["prime"])        # retire une VARIABLE
```

Deux opérations, deux pertes de nature différente.

- Retirer des **lignes** réduit l'échantillon, et le déforme si les manquants ne sont pas au hasard.
- Retirer une **colonne** conserve tout l'échantillon, mais vous prive d'une variable — peut-être celle dont vous aviez besoin.

Une variable manquante à 60 % n'est pas récupérable. Une variable manquante à 3 % ne justifie pas de perdre 3 % de l'échantillon si elle ne sert pas à votre question.

La question n'est pas « combien manque-t-il ? » mais « en ai-je besoin ? ».

Regarder avant de décider

Aucun tableau de nombres ne remplace un coup d'œil sur la distribution.

```
df["revenu"].hist(bins=50)           # la forme d'ensemble
df.boxplot(column="revenu")          # les points isolés
df.plot.scatter(x="age", y="revenu")  # les points hors logique
```

- L'**histogramme** révèle une distribution tronquée — un plafond de diffusion — un pic anormal, ou une bimodalité qui trahit deux populations mélangées.
- La **boîte à moustaches** isole les points éloignés, sans dire s'ils sont faux.
- Le **nuage de points** montre ceux qui contredisent la logique économique — un salaire élevé à 16 ans.

Nous reviendrons sur les graphiques en séance 6. Ici ils servent à **diagnostiquer**, pas à communiquer.

Imputer par la moyenne : ce que cela casse

```
df["salaire"] = df["salaire"].fillna(df["salaire"].mean())
```

Cette ligne est simple, courante, et abîme les données de trois façons :

- la **variance** diminue — on a ajouté des observations sans dispersion ;
- les **erreurs-types** sont sous-estimées : le calcul croit disposer de plus d'information qu'il n'en existe réellement ;
- les **corrélations** avec les autres variables sont diluées vers zéro.

Imputer, c'est faire une hypothèse sur ce qu'on ne voit pas. C'est un modèle, pas une réparation — et un modèle se déclare.

Ce que nous ferons dans ce cours

1. **Conserver et signaler** : ne rien retirer, et rapporter le nombre d'observations effectivement utilisées pour chaque statistique. C'est la colonne « Observations » de tout tableau d'article.
2. **Documenter** : dire combien de lignes sont concernées et sur quelles variables.
3. **Tester la sensibilité** : refaire le calcul principal sous deux hypothèses, et regarder si la conclusion tient.

Un résultat qui change de signe selon le traitement des manquants n'est pas un résultat. Un résultat qui tient sous les deux hypothèses est solide, et le dire renforce l'argument.

Écrire votre première fonction

Une règle de nettoyage appliquée trois fois s'écrit une fois.

```
def nettoyer_montant(serie, codes_manquants):  
    # Convertit en numerique, puis met les codes en manquant  
    s = pd.to_numeric(serie, errors="coerce")  
    return s.replace(codes_manquants, np.nan)  
  
df["salaire"] = nettoyer_montant(df["salaire"], [-1, 999999])  
df["prime"]   = nettoyer_montant(df["prime"],   [-1, 999999])
```

Trois bénéfices : la règle est écrite une seule fois, elle porte un nom qui dit ce qu'elle fait, et la corriger la corrige partout.

Copier-coller trois fois la même ligne, c'est créer trois occasions de se tromper.

Les doublons

Deux questions différentes

```
df.duplicated().sum()           # lignes identiques partout
df.duplicated(subset=["id"]).sum() # identifiant repete
```

- Une ligne **entièrement identique** à une autre est presque toujours une erreur : import répété, fusion mal faite.
- Un **identifiant répété** n'est pas forcément une erreur. Tout dépend de l'unité statistique.

Dans un panel, chaque individu apparaît une fois par année : la clé est le couple (individu, année). Dans un fichier de transactions immobilières, un même logement peut être vendu deux fois.

```
doublons = df[df.duplicated(subset=["id"], keep=False)]  
doublons.sort_values("id").head(20)
```

Avec `keep=False`, toutes les occurrences sont affichées, pas seulement les répétitions. On peut alors voir en quoi les lignes diffèrent.

Le diagnostic

Si les lignes sont identiques partout : doublon technique, à supprimer. Si elles diffèrent sur une date, un montant, un statut : ce sont des observations distinctes, et l'unité statistique n'est pas celle que vous croyiez.

Documenter

Ne jamais écraser les données brutes

```
brut = pd.read_csv("salaires.csv", sep=";")    # jamais modifie  
df    = brut.copy()                            # on travaille ici
```

Deux raisons.

- Vous vous tromperez, et il faudra revenir en arrière sans tout retélécharger.
- La comparaison brut / nettoyé est ce qui vous permet de *mesurer* l'effet de vos décisions.

Étape	Décision	Lignes	Justification
Chargement	—	84 320	
Codes 99 → manquant	Conversion	84 320	Dictionnaire, p. 14
Doublons techniques	Suppression	84 102	Lignes identiques partout
Salaires nuls	Conservation	84 102	Non-salariés, hors champ
Âge hors 15–64	Suppression	71 455	Champ de l'étude

Ce tableau tient en cinq lignes de notebook et rend votre travail vérifiable. Il constitue aussi, presque tel quel, la note méthodologique de votre mémoire de M1.

Une décision non documentée est une décision cachée.

Ce qu'un article ne montre pas

Tout article empirique commence par un tableau de statistiques descriptives.

	Moyenne	Observations
Salaire mensuel net	2 180	71 455
Âge	39,4	71 455
Part de diplômés du supérieur	0,42	68 902

- Pourquoi la dernière ligne compte-t-elle moins d'observations ?
- Où sont passées les 12 865 personnes du fichier initial ?
- Les salaires nuls ont-ils été conservés ?

Aucune réponse n'est dans le tableau. Toutes sont dans la note méthodologique — si elle existe.

Trois questions avant de passer à l'analyse

1. Combien de lignes ai-je perdues, et en quoi diffèrent-elles de celles que je garde ?
2. Chacune de mes décisions est-elle justifiable à voix haute devant quelqu'un qui conteste ?
3. Mon résultat principal survivrait-il à la décision inverse ?

La troisième est celle qui distingue un travail solide d'un travail plausible. C'est aussi une question type du contrôle continu.

Pour la suite

- Repérer les non-réponses codées en chiffres, et les convertir
- Distinguer non-réponse et hors champ — deux choses différentes
- Documenter chaque décision, et mesurer son effet sur le résultat

Le livrable n'est pas un fichier propre : c'est le **journal des décisions**, avec l'effet chiffré de chacune.

1. Terminez votre journal de nettoyage.
2. Recalculez le salaire moyen après chaque étape et notez l'écart.
3. Vérifiez que le notebook s'exécute de haut en bas.
4. Récupérez le notebook de la séance 4 sur le site du cours.

`stefaniamarccassa.github.io/analyse_des_donnees`

Des questions ?